

**Debreceni SZC Mechwart András  
Gépipari és Informatikai Technikum**



Szoftverfejlesztő és -tesztelő

Vizsgaremek

Étteremkereső Webalkalmazás

Készítette: Villás Attila  
Czövek Dominik  
Földi Soma

2026.

## Tartalomjegyzék

---

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Bevezetés.....                                     | 2  |
| 2. A projekt áttekintése és funkciói.....             | 4  |
| 3. Csapatmunka megvalósítása .....                    | 6  |
| 4. Felhasználói dokumentáció .....                    | 9  |
| 5. Fejlesztői dokumentáció.....                       | 16 |
| 6. Az alkalmazás tesztelése és a hibák javítása ..... | 26 |
| 7. Összegzés.....                                     | 29 |
| 8. Továbbfejlesztési lehetőségek.....                 | 30 |
| 9. Irodalomjegyzék .....                              | 31 |
| 10. Mellékletek .....                                 | 32 |

# 1. Bevezetés

---

## 1.1. A témaválasztás indoklása

Az Étteremkereső Webalkalmazás célja egy olyan modern, könnyen kezelhető webes rendszer létrehozása volt, amely valódi hétköznapi problémára ad használható választ. A legtöbb emberrel előfordult már, hogy szeretett volna gyorsan találni egy jó helyet ebédhez vagy vacsorához, de a választást egyszerre befolyásolta a távolság, az árkategória, a konyhatípus és mások visszajelzése is. A projekt erre a döntési helyzetre kínál egy átláthatóbb, kényelmesebb megoldást.

A téma vizsgaremekként azért bizonyult jó választásnak, mert egyszerre közérthető és szakmailag is kellően összetett. A kész rendszer könnyen bemutatható, ugyanakkor a háttérben több fontos technológiai terület kapcsolódik össze: felhasználókezelés, hitelesítés, adatbázis-műveletek, térképes megjelenítés, képkezelés, szűrés, értékelési logika és kliens–szerver kommunikáció. Így a projekt nemcsak látványos, hanem fejlesztői szempontból is tartalmas feladat volt.

Ez a dokumentáció az alkalmazás felépítését, működését és a fejlesztés során elért eredményeket mutatja be. A leírás a repository szerkezetére, a megvalósított funkciókra és a jelenlegi állapotra támaszkodik, ezért a hangsúly mindenhol a ténylegesen elkészült megoldásokon van, nem pedig pusztán terveken vagy elképzeléseken.

Szakmai oldalról a téma azért volt különösen hasznos, mert az étteremkeresés jóval több, mint egy egyszerű lista megjelenítése. A felhasználók szeretnének kedvenceket menteni, véleményeket olvasni, saját értékelést írni, térképen tájékozódni és személyre szabott ajánlásokat kapni. Egy ilyen rendszer jól modellezi a valódi webalkalmazások tipikus felépítését, és lehetőséget adott a kliensoldali állapotkezelésre, a JWT-alapú hitelesítésre, a relációs adatbázisok és a REST-szerű API-k gyakorlati alkalmazására is.

A kitűzött célokat két nagy csoportra lehet bontani. Felhasználói oldalról fontos volt, hogy lehessen regisztrálni és bejelentkezni, éttermeket böngészni, részletes étteremoldalt megnyitni, képeket és véleményeket megjeleníteni, kedvenceket kezelni, valamint preferenciák alapján ajánlásokat kérni. Fejlesztői oldalról ugyanolyan lényeges szempont volt, hogy a projekt tagolt, bővíthető és önállóan futtatható legyen, valamint adatbázis-szinten is koherens maradjon, és tartalmazzon alapvető tesztelési, illetve hibakezelési megoldásokat.

A dokumentáció további felépítése ennek megfelelően először bemutatja a rendszer funkcionális hátterét, majd kitér a csapatmunka megvalósítására, a felhasználói dokumentációra, ezt követően a fejlesztői megközelítésre, a technológiai döntésekre, az adatmodellre, a tesztelésre, végül pedig összegzi a projekt tapasztalatait és a továbbfejlesztési lehetőségeket.

## 1.2. Kitűzött célok

A projekt legfontosabb funkcionális célja egy olyan webes alkalmazás létrehozása volt, amelyben a felhasználó egyszerűen eligazodik, és néhány kattintással eljut a számára releváns éttermekhez. Ehhez szükség volt a listázási lehetőségre, a részletező nézetre, a térképes keresésre, a kedvencek kezelésére, valamint az értékelések és vélemények támogatására.

A személyre szabás külön célként jelent meg. A repository felépítése és a kliensoldali oldalak működése alapján jól látszik, hogy a rendszer nem csak általános étteremlistát kíván megjeleníteni, hanem a felhasználó által megadott preferenciákat is be szeretné vonni az ajánlásba.

Ez egy vizsgaremek esetében fontos pluszérték, mert megmutatja, hogy a projekt túlmutat egy egyszerű CRUD-alkalmazás szintjén.

Fejlesztői oldalról a célok között szerepelt a frontend és a backend logikai szétválasztása, a tiszta API-réteg kialakítása, a fejlesztői környezetek egyszerű használata, valamint az adatbázis automatikus előkészítésének megoldása. A későbbi bővíthetőség érdekében külön előny, hogy a projekt nem egyetlen monolitikus fájlhalmazként épül fel, hanem jól elkülönített frontend, backend, database és tests részstruktúrára bontható.

| Célterület       | Részletes cél                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------|
| Felhasználói cél | A felhasználó gyorsan találjon megfelelő éttermet, menthesse kedvenceit és kezelhesse értékeléseit. |
| Személyre szabás | A preferenciák ténylegesen befolyásolják az ajánlási és megjelenítési logikát.                      |
| Fejlesztői cél   | A projekt legyen jól tagolt, önállóan futtatható és bővíthető.                                      |
| Demózhathóság    | Az alkalmazás rövid előkészítéssel, előre seedelt adatokkal is elindítható legyen.                  |

### 1.3. A dokumentáció felépítése

A dokumentáció a továbbiakban először a rendszer funkcionális hátterét és a projekt szervezési logikáját ismerteti, majd külön felhasználói és fejlesztői nézőpontból is bemutatja az alkalmazást. Ezt követi a tesztelés és hibajavítás fejezete, végül az összegzés, a továbbfejlesztési lehetőségek, valamint a bibliográfiai és mellékleti rész.

## 2. A projekt áttekintése és funkciói

### 2.1. A megoldandó probléma

Az Étteremkereső Webalkalmazás lényege, hogy a felhasználó központi helyen tudja kezelni az étteremkereséssel kapcsolatos alapvető tevékenységeit. A rendszer nem pusztán felsorol éttermeket, hanem a különböző nézetek és műveletek összekapcsolásával támogatja a döntést. A felhasználó a főoldalról kiindulva böngészheti az elérhető éttermeket, térképen is láthatja őket, megnyithatja az egyes részletező oldalakat, elolvashatja mások visszajelzéseit, illetve saját véleményét is rögzíthet.

A megoldás erőssége, hogy egyetlen rendszerben kezeli a böngészést, a személyes beállításokat és az interakciókat. A kedvencek funkció segítségével a felhasználó hosszabb távon is építhet saját gyűjteményt, a profiloldalon pedig olyan preferenciákat állíthat be, amelyek alapján a rendszer személyre szabott ajánlásokat jelenít meg. A térképes nézet különösen hasznos abból a szempontból, hogy a választás földrajzi dimenzióját is láthatóvá teszi.

Az alkalmazás várható célcsoportja széles. Használhatja olyan felhasználó, aki egy adott városban belül gyorsan szeretne új helyet keresni, de olyan is, aki inkább a kedvencei és korábbi értékelései alapján szeretné rendszerezni saját tapasztalatait. Oktatási projektként a rendszer azért különösen jól választott téma, mert egyszerre könnyen bemutatható és mégis tartalmaz elegendő mélységet a komplex webalkalmazások jellemző problémáinak modellezéséhez.

Nem funkcionális oldalról a legfontosabb elvárások közé tartozott a logikus felépítés, a stabil indítás, az átlátható kódszerkezet és a felhasználóbarát működés. A commitelőzményből is látszik, hogy a végső szakaszban hangsúlyt kapott a validáció, a startup seedelés, a szűrés stabilizálása és a hibakezelés, vagyis a projekt nem csupán funkciókat halmozott fel, hanem a használhatóság és megbízhatóság irányába is finomodott.

### 2.2. Fő funkcionális elemek

| Funkcióterület    | Megvalósított képesség                                                         |
|-------------------|--------------------------------------------------------------------------------|
| Felhasználói fiók | Regisztráció, bejelentkezés, kijelentkezés, felhasználói adatok visszatöltése. |
| Preferenciák      | Kezdeti preferenciák megadása és későbbi módosítása a profiloldalon.           |
| Étteremlista      | Éttermek böngészése, keresése és térképes megjelenítése.                       |
| Részletező oldal  | Egy étterem adatainak, képeinek és értékeléseinek megjelenítése.               |
| Kedvencek         | Éttermek mentése személyes listába.                                            |
| Értékelések       | 1–5-ös skála és szöveges vélemény rögzítése.                                   |
| Ajánlások         | Preferenciák alapján személyre szabott étteremlista.                           |
| Képek             | Éttermhez tartozó képek listázása és feltöltése.                               |

### 2.3. Nem funkcionális elvárások

| Szempont    | Magyarázat                                             |
|-------------|--------------------------------------------------------|
| Modularitás | A frontend és backend fizikai és logikai elkülönítése. |

|                     |                                                                       |
|---------------------|-----------------------------------------------------------------------|
| Átláthatóság        | Funkciók szerinti controller- és API-modulbontás.                     |
| Futtathatóság       | Automatikus adatbázis-előkészítés és fejlesztői indítás támogatása.   |
| Biztonság           | JWT alapú hitelesítés, jelszószabályok, védett útvonalak.             |
| Felhasználói élmény | Térképes megjelenítés, kedvencek, hibakezelés, rezponzív gondolkodás. |

## 2.4. A rendszer célcsoportja és felhasználási helyzetei

A rendszer elsődleges célcsoportja a városon belül új éttermet kereső felhasználó, de ugyanilyen jól használható olyan helyzetekben is, amikor a felhasználó már korábbi kedvencekből válogat, vagy a saját preferenciái alapján szeretne új ajánlásokat kapni. Oktatási szempontból a projekt tipikus teljes webalkalmazásnak tekinthető, ezért jó mintát ad arra, hogyan kapcsolódnak össze a kliensoldali felület, a backend API és a relációs adattárolás.

## 3. Csapatmunka megvalósítása

### 3.1. Verziókezelés és munkaszervezés

A csapatmunka egyik legfontosabb alapját a közös GitHub repository adta. A fejlesztés kisebb, jól követhető lépésekben haladt, így az új funkciók, a hibajavítások és a finomítások is elkülöníthetők maradtak. Ez nemcsak az átláthatóságot segítette, hanem azt is, hogy bármikor vissza lehessen követni, mikor milyen változtatás került be a projektbe.

A fejlesztés során több jól elkülöníthető feladatkör rajzolódott ki. Külön munkát jelentett a frontend felületek kialakítása, ezen belül a főoldal, az étteremlista, a regisztráció, a bejelentkezés, a kedvencek és a profiloldal elkészítése. Ezzel párhuzamosan épült a backend API, az autentikáció, a CORS beállítás, az adatseedelés, valamint a szűrési és validációs logika. A későbbi szakaszban ehhez kapcsolódott a képkezelés és az egyes modulok összehangolása is.

A repository szerkezete jól támogatta a párhuzamos munkavégzést. A backend, a frontend és az adatbázishoz kapcsolódó részek külön mappákba kerültek, ezért a feladatok könnyebben szétválaszthatók voltak. Ez az integrációt is megkönnyítette, mert a kliensoldali és szerveroldali munka végül egy egységes rendszerben találkozott össze.

A projektből az is jól látszik, hogy egy ilyen webalkalmazásnál a feladatokat nem lehet teljesen elszigetelten kezelni. A térképes megjelenítés, a képkezelés vagy az értékelések egyszerre érintették a frontendet, a backendet és az adatmodellt is. Emiatt a folyamatos egyeztetés a fejlesztés természetes része volt.

A feladatmegosztás a projekt szerkezete és a fejlesztési irányok alapján jól összefoglalható. A táblázat azt mutatja meg, hogy a munka mely területek köré szerveződött, és hogyan lehetett logikusan szétosztani a frontend, backend, integrációs és dokumentációs feladatokat.

### 3.2. A fejlesztés főbb szakaszai

| Fejlesztési szakasz        | Tartalom                                                              | Jelentőség                                                |
|----------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------|
| Tervezés és alap koncepció | README, funkciólista, mappastruktúra, adatmodell-vázlat               | A projekt céljainak és rétegeinek rögzítése.              |
| Backend alapok             | API-projekt, adatbázis-kapcsolat, hitelesítés és CORS előkészítése    | A szerveroldali mag megteremtése.                         |
| Frontend alapok            | Főoldal, navigáció, étteremlista, login és regisztráció első verziója | A felhasználói felület működő alapjainak kialakítása.     |
| Adatkezelés és seedelés    | Mintaadatok, cím és koordináták, listázás és térképes lekérdezések    | A bemutathatóság és tesztelhetőség javítása.              |
| Felhasználói funkciók      | Kedvencek, értékelések, profil és védett route-ok                     | A személyre szabott használat támogatása.                 |
| Térkép és média            | Térképes nézet, étteremképek, részletező oldal bővítése               | A projekt látványos és komplex részeinek összekapcsolása. |

|              |                                                                 |                                                |
|--------------|-----------------------------------------------------------------|------------------------------------------------|
| Stabilizálás | Validáció, szűrés, hibakezelés, induláskori seedelés finomítása | A leadható, megbízható végállapot kialakítása. |
|--------------|-----------------------------------------------------------------|------------------------------------------------|

### 3.3. Feladatmegosztás

| Név            | Fő terület                            | Rövid leírás                                                   |
|----------------|---------------------------------------|----------------------------------------------------------------|
| Villás Attila  | Backend és adatelérés                 | API-végpontok, hitelesítés és adatbázis-logika.                |
| Czövek Dominik | Frontend és felhasználói felület      | Oldalak, routing és kliensoldali integráció.                   |
| Földi Soma     | Integráció, dokumentáció és tesztelés | Modulok összehangolása, ellenőrzés és dokumentációs feladatok. |

### 3.4. A csapatmunka tanulságai

A committörténet alapján egyértelmű, hogy nem lineáris, egyszemélyes fejlesztés zajlott, hanem több részterület haladt egymás mellett. A munka közben természetesen voltak döcögősebb szakaszok, kisebb nézeteltérések és olyan pontok is, amikor ugyanarra a problémára többféle megoldás merült fel. Ezek azonban végül nem hátráltatták, hanem inkább pontosították a közös irányt: az egyeztetések után sikerült összehangolni az elképzeléseket, és a rendszer stabilabb lett tőlük. A legfontosabb tanulság az volt, hogy a korai moduláris szétválasztás, vagyis a frontend, backend, adatbázis és teszt rétegek külön kezelése jelentősen megkönnyíti a későbbi integrációt. Ugyanakkor a projekt vége felé mindig kiemelt figyelmet kell kapniuk a keresztmetszeti hibáknak, például a CORS-nak, a validációnak, a bootstrap állapotnak, a route-védelemnek és a seedelésnek.



## 4. Felhasználói dokumentáció

### 4.1. Rendszerkövetelmények

| Elem                      | Ajánlott érték                                                            | Magyarázat                                                                                    |
|---------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Operációs rendszer        | Windows 10/11, Linux vagy macOS                                           | A frontend és a backend parancssoros indításához bármely modern fejlesztői rendszer megfelel. |
| Backend futtatókörnyezet  | .NET 8 SDK                                                                | Az API projekt ASP.NET Core alapú, a jelenlegi kódrendszer .NET 8-at használ.                 |
| Frontend futtatókörnyezet | Node.js 20+ és npm                                                        | A Vite és React környezet kezeléséhez szükséges.                                              |
| Adatbázis                 | SQLite                                                                    | Külön adatbázis-szerver nélkül használható, könnyen hordozható megoldás.                      |
| Böngésző                  | Chrome, Edge vagy Firefox aktuális verzió                                 | A webalkalmazás felhasználói felületének kipróbálásához.                                      |
| Portok                    | Frontend: 5173 / 4173,<br>Backend: alapértelmezés szerint 5000 környezete | A frontend kódja a VITE_API_BASE_URL hiányában a http://localhost:5000/api végpontot várja.   |

### 4.2. A telepítés előkészítése

Az alkalmazás futtatásához a backend és a frontend részt külön kell elindítani. A backend ASP.NET Core alapú API, a frontend pedig Vite-re épülő React alkalmazás, ezért a futtatási környezetben .NET SDK-ra és Node.js-re is szükség van. A gyakorlatban ez azt jelenti, hogy két terminálablakkal érdemes dolgozni: az egyikben a backend fut, a másikban pedig a frontend fejlesztői szervere.

A repository felépítése alapján a backend külön projektmappában található. Az indítás tipikus menete egyszerű: a megfelelő mappába belépve először a függőségek visszaállítása történik meg, majd maga a projekt fut. A kódban látható induláskori adatbázis-előkészítés miatt a rendszer első indításkor automatikusan létrehozhatja vagy frissítheti az adatbázist, majd seedelheti a szükséges alapadatokat is. Ez demó és tesztelés közben különösen kényelmes, mert nincs szükség külön kézi SQL-importra.

A frontend indítása hasonlóan egyszerű: a Vite-projekt mappájában az npm csomagok telepítése után elindítható a fejlesztői szerver. A kliensoldali Axios beállítás szerint az alkalmazás alapértelmezésben a http://localhost:5000/api címen várja a backend API-t. Ha a backend ettől eltérő címen vagy porton fut, akkor ezt a VITE\_API\_BASE\_URL környezeti változóval lehet felülírni, különben a frontend nem fogja elérni az API-t.

A használatához érdemes először a backendet elindítani, és csak utána a frontendet. Ha a frontend hamarabb indul el, akkor a listaoldal, a bejelentkezés vagy a részletező nézetek hibát jelezhetnek, mert még nem érik el az API-t. Demó előtt ezért célszerű gyorsan ellenőrizni, hogy mindkét szolgáltatás a megfelelő porton fut-e.

*Backend indítási minta*

```
cd backend/RestaurantFinder.Api
dotnet restore
dotnet run
```

### Frontend indítási minta

```
cd etteremkereso-frontend
npm install
npm run dev
```

### Opionális környezeti beállítás

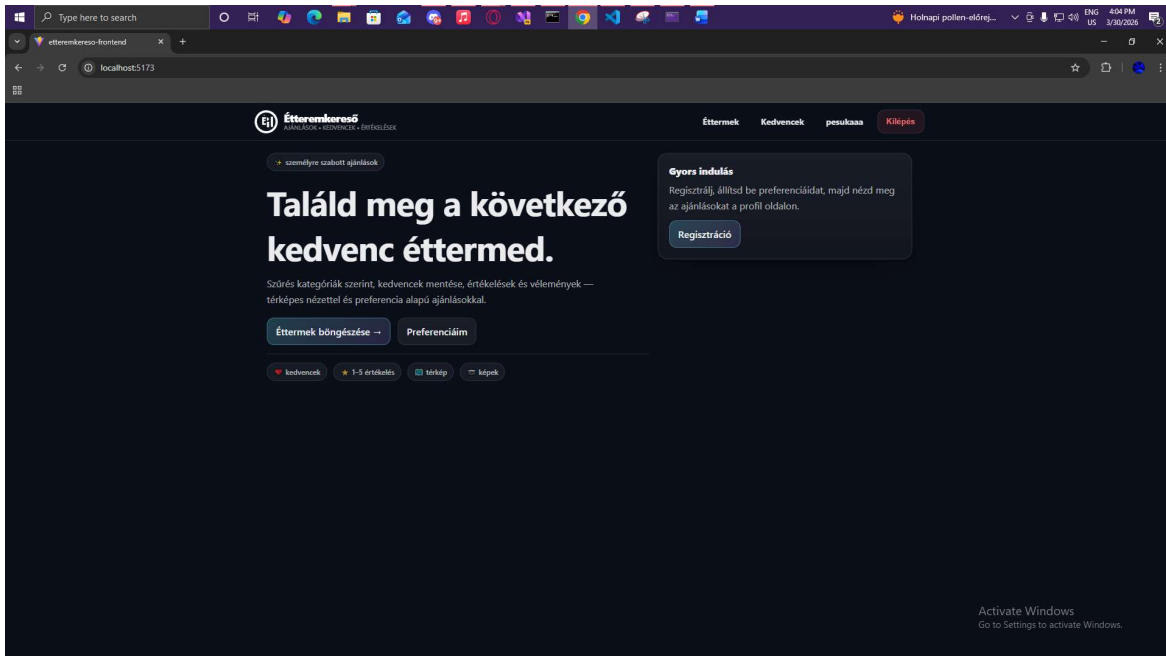
```
VITE_API_BASE_URL=http://localhost:5000/api
```

## 4.3. Az alkalmazás használata képernyőképekkel illusztrálva

### 4.3.1. Főoldal

A főoldal elsődleges szerepe, hogy rövid idő alatt átadja az alkalmazás értékajánlatát. A feliratok és gyorsindító elemek alapján azonnal látható, hogy az oldal nem pusztán étteremlistázó felület, hanem kedvenceket, értékeléseket és ajánlásokat is kezelő rendszer. Ez jó belépési pont a felhasználónak, mert a projekt célja már az első képernyőn világosan kommunikálódik.

A főoldalról tipikusan két irányba indulhat el a felhasználó: vagy böngészi az éttermeket, vagy regisztrál / bejelentkezik a személyre szabott funkciók használatához. Az ilyen kettős indulási lehetőség kényelmes, mert az alkalmi látogatót és a visszatérő, bejelentkezett felhasználót is kiszolgálja.

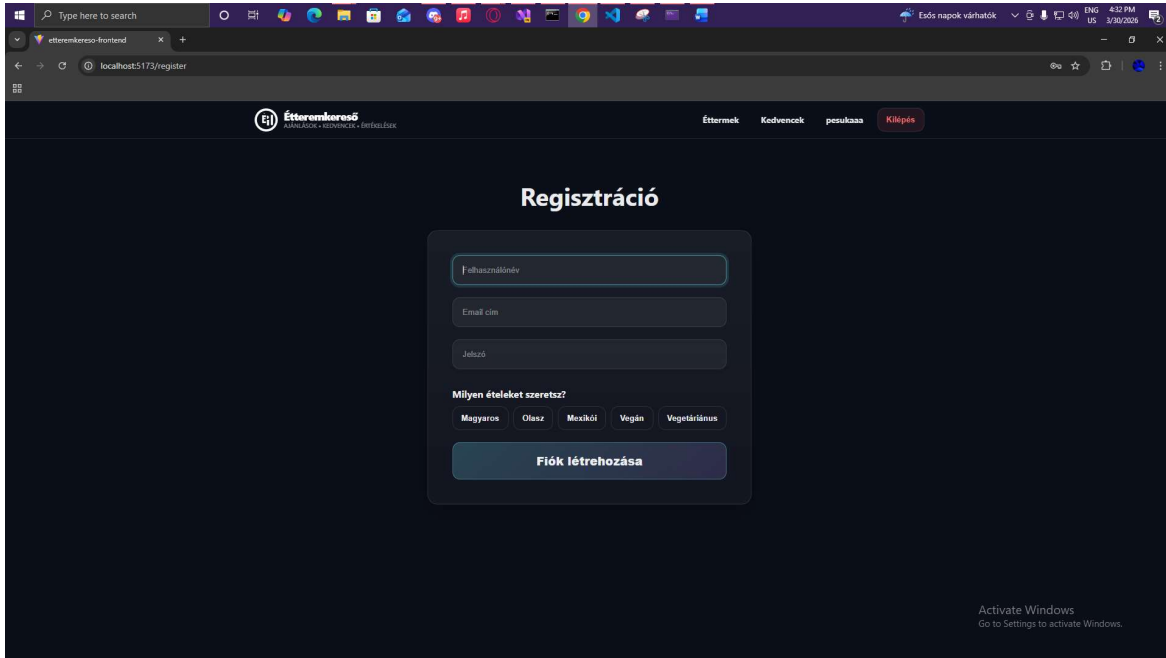


1. ábra - A főoldal kezdőnézete

### 4.3.2. Regisztráció

A regisztrációs felület a szokásos azonosító adatokon túl azért érdekes, mert már ezen a ponton képes bevonni a felhasználói preferenciákat is. Ez jól illeszkedik a projekt fő koncepciójához: a rendszer már a belépés pillanatától személyre szabott élményt szeretne kialakítani. A felületen a preferenciák csoportokba rendezve jelenhetnek meg, ami javítja az átláthatóságot és gyorsítja a kitöltést.

A dokumentációban ezt a képernyőképet azért érdemes külön kiemelni, mert jól szemlélteti a frontend és a backend együttműködését. A preferenciák listája ugyanis nem statikus díszítőelem, hanem a szerverről betöltött, majd a felhasználóhoz kapcsolt adatstruktúra része.

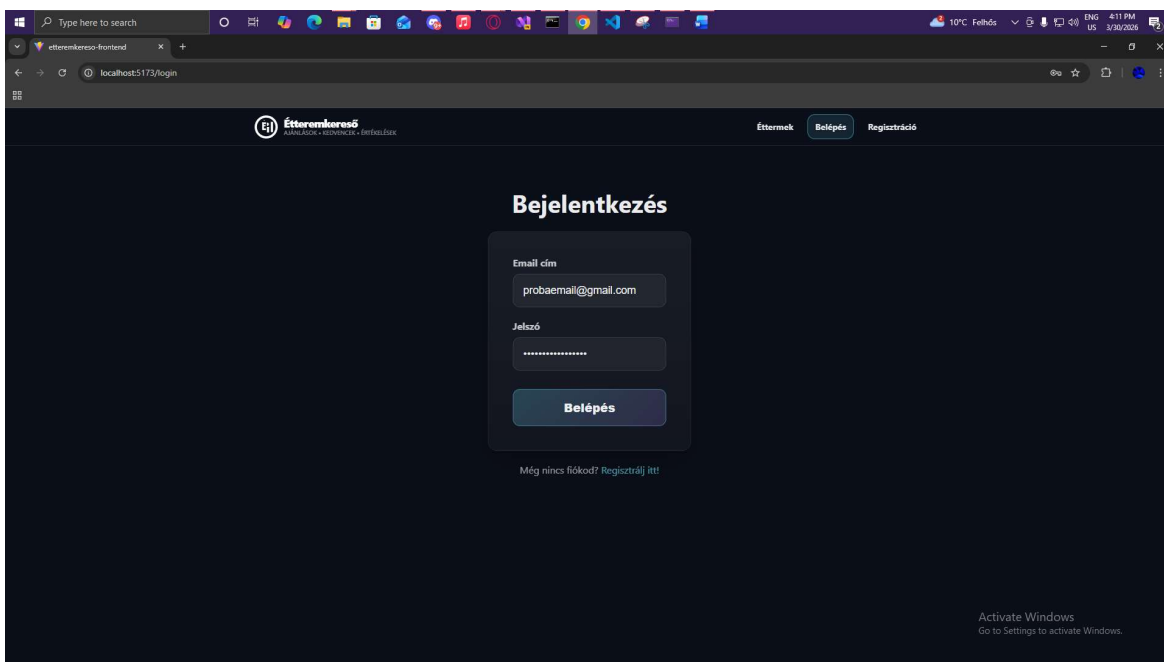


2. ábra - Regisztrációs felület preferenciaválasztással

### 4.3.3. Bejelentkezés

A bejelentkezési oldal a lehető legegyszerűbb felhasználói élményt célozza: a felhasználó e-mail címmel és jelszóval léphet be. A commitelőzmény alapján a hibakezelés külön fejlesztési pontként jelent meg, ami arra utal, hogy a csapat nem elégedett meg a technikailag működő, de kommunikáció nélküli formával, hanem a felhasználó számára is értelmezhető visszajelzést szeretett volna adni.

Sikeres belépés után a kliensoldal a token tárolásával és a saját felhasználói adatok újratöltésével vált át bejelentkezett állapotba. Ez az oldal tehát a teljes autentikációs folyamat belépési kapuja.

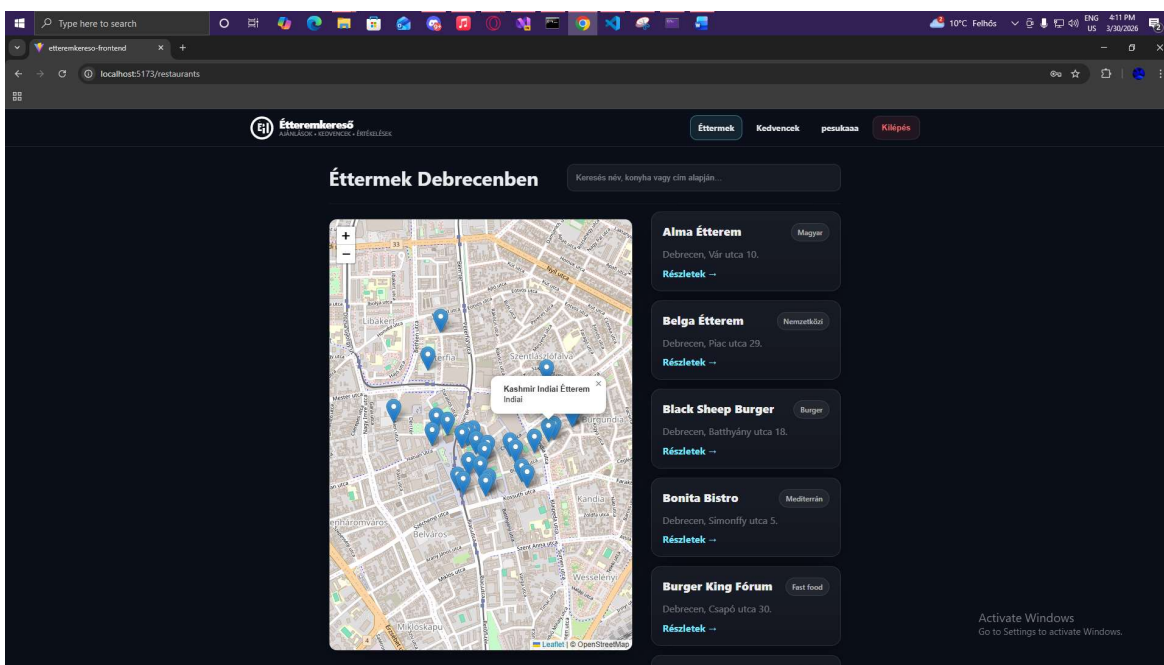


3. ábra - Bejelentkezési felület

#### 4.3.4. Éttermek listája és térképes nézet

Az éttermek listája a rendszer egyik legfontosabb központi oldala. Itt a felhasználó egyszerre kap listás és térképes áttekintést, ami különösen hasznos akkor, ha nem konkrét név alapján keres, hanem inkább a helyszín és a kategória együttese érdekli. A keresőmezővel gyorsan szűkíthető a megjelenő találatok köre.

A térképes nézet oktatási szempontból is erős elem. Nem pusztán díszítő funkció, hanem valódi geoadatokkal dolgozik, és marker-alapon jeleníti meg az éttermeket. Ezzel a projekt kilép az egyszerű listaoldalak világából, és egy kifejezetten látványos, mégis hasznos funkciót mutat be.

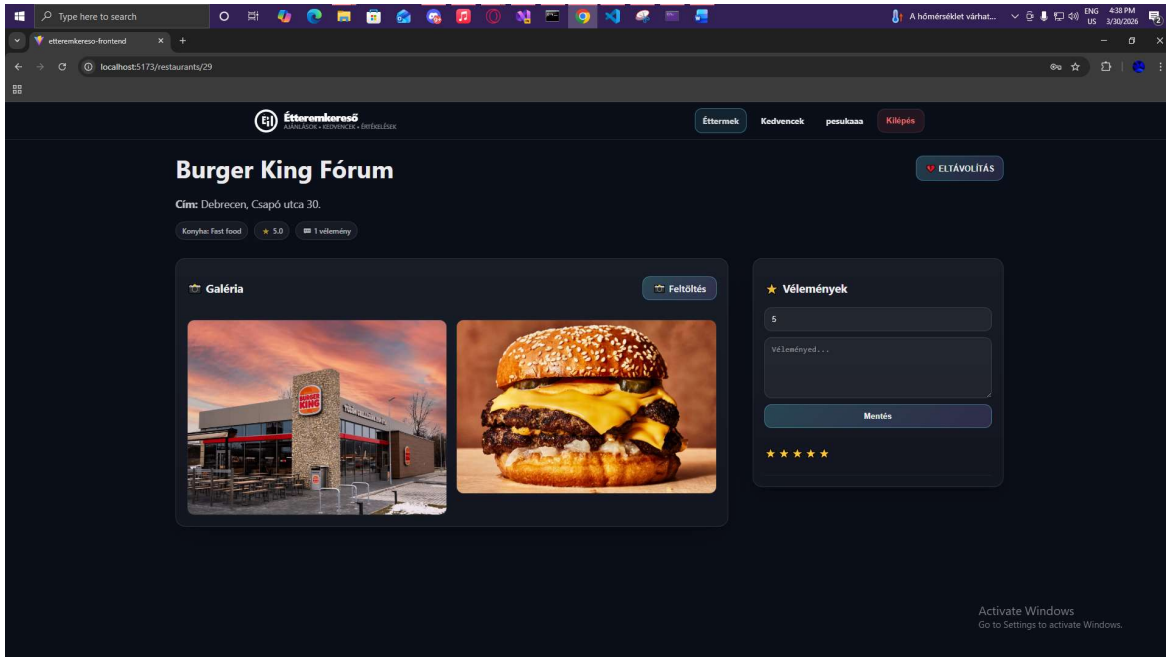


4. ábra - Étteremlista és térképes nézet

### 4.3.5. Étterem részletező oldal

Az étterem részletező oldal a rendszer egyik legkomplexebb felülete. Itt egyszerre jelennek meg az alapadatok, az átlagos értékelés, a képek, a felhasználói vélemények, valamint a kedvencek és az új értékelés kezeléséhez szükséges interakciók. Jó példa ez arra, amikor egy komponens több API-forrásból épít fel egy összetett nézetet.

A bejelentkezett felhasználó számára ez az oldal már valódi interakciós központ. Innen lehet kedvenchez adni az éttermet, képeket megtekinteni, illetve saját pontszámot és szöveges véleményt rögzíteni. A részletező nézet jól mutatja, hogy a projekt célja nem csupán információk megjelenítése, hanem a közösségi jellegű visszajelzések támogatása is.

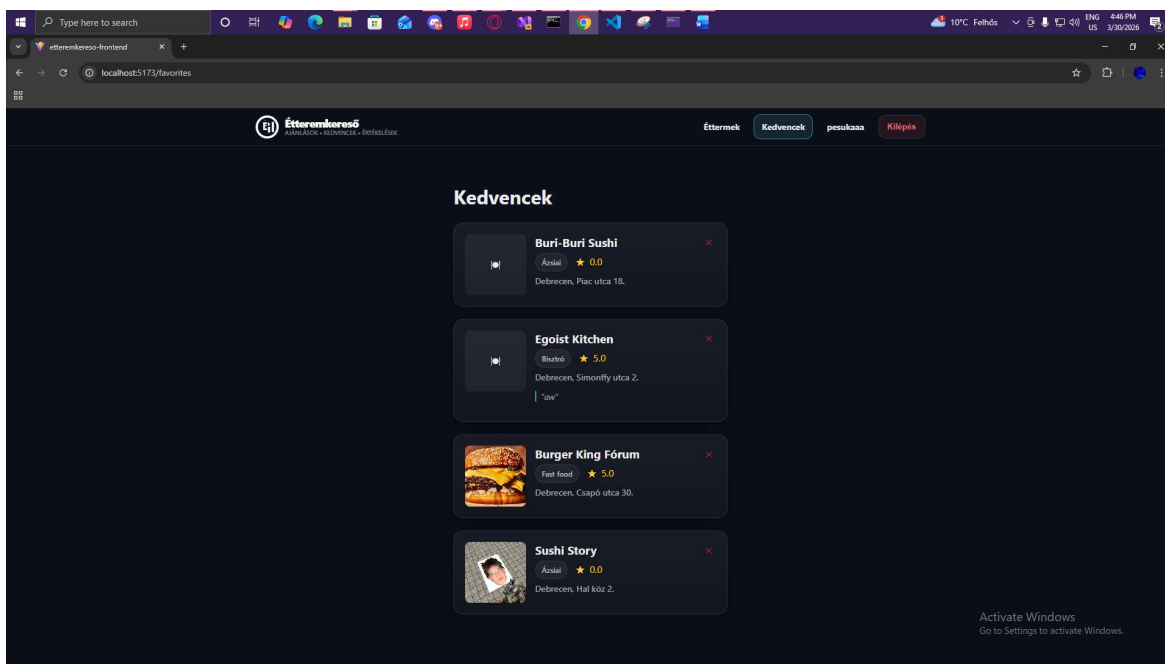


5. ábra - Étterem részletező oldal

### 4.3.6. Kedvencek

A Kedvencek oldal a felhasználó személyes gyűjteményének felel meg. Funkcionálisan azért fontos, mert a böngészés során kiválasztott éttermek nem vesznek el, hanem később is gyorsan előhívhatók. Egy étteremkereső rendszerben ez kifejezetten praktikus funkció, hiszen a felhasználó ritkán dönt azonnal; gyakran szeretné összehasonlítani a korábban elmentett lehetőségeket.

Fejlesztői nézőpontból ez az oldal jó példa a védett útvonalak használatára. Mivel a kedvencek természetüknél fogva felhasználóhoz kötött adatok, az oldal csak hitelesített állapotban jelenhet meg.

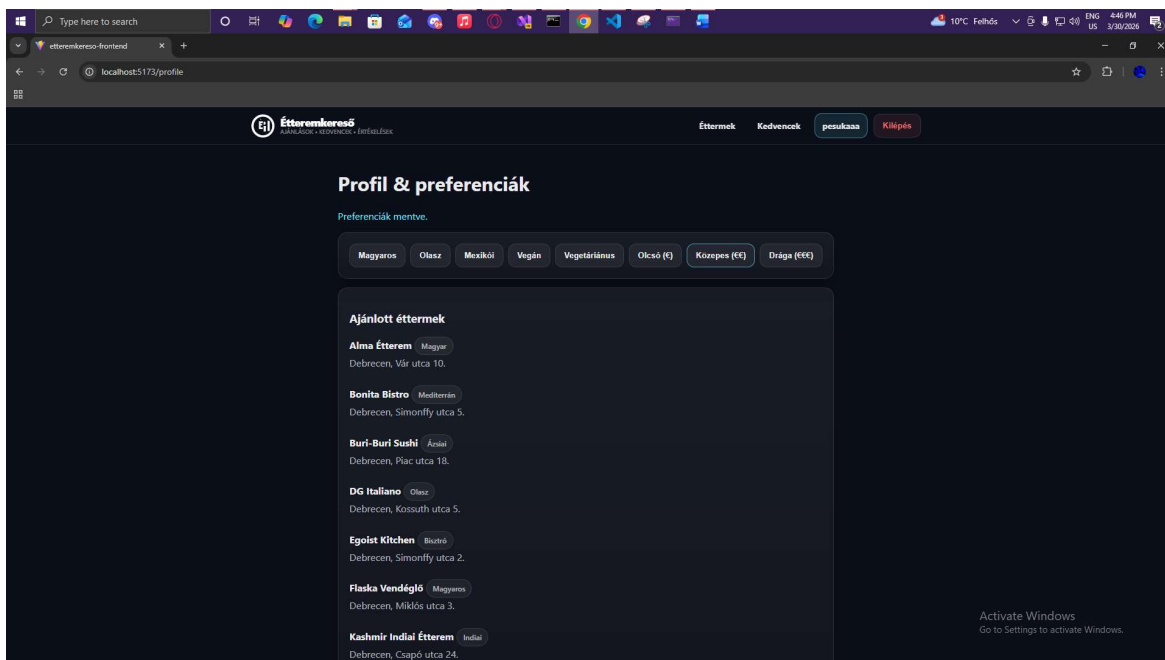


6. ábra - Kedvencek oldal

#### 4.3.7. Profil és ajánlások

A Profil oldal a személyre szabás központi helye. Itt a felhasználó nemcsak a saját azonosító adataihoz kapcsolódó nézetet kapja vissza, hanem preferenciákat kezelhet, és azok alapján új ajánlásokat kérhet. Ez a projekt egyik legerősebb funkcionális üzenete: a rendszer nem azonos módon jelenik meg mindenki számára, hanem a felhasználói beállításokhoz igazodik.

A profiloldal fejlesztői szempontból azért különösen érdekes, mert egyszerre demonstrál szerveroldali adatlekérést, állapotfrissítést és újralekérést. Az ajánlások refetch logikája és a preferenciák ki-bekapcsolása jól mutatja, hogy a kliensoldal aktívan együttműködik a backenddel.



7. ábra - Profiloldal és ajánlási nézet

#### 4.4. Tipikus felhasználói forgatókönyv

A használat tipikus menete a főoldalon kezdődik. Innen a felhasználó az éttermek böngészése felé haladhat tovább, vagy először regisztrálhat, hogy személyre szabottabb élményt kapjon. A főoldal vizuálisan is hangsúlyozza a projekt fő értékeit: ajánlások, kedvencek, értékelések, térképes nézet és képek.

Regisztráció után a felhasználó beállíthatja preferenciáit. A kliensoldali kód alapján a preferenciák logikailag csoportosítva vannak, például konyhatípus és árkategória szerint. Ez arra utal, hogy a rendszer a személyre szabást nem utólagos kiegészítő funkciónak tekinti, hanem már a belépési folyamat részévé teszi.

Az éttermek oldal a keresés és a térképes böngészés kombinációját nyújtja. A felhasználó egyszerre látja a listát és a marker-alapú térképet, ami különösen hasznos akkor, ha egy földrajzi területen belül szeretne választani. A részletező oldal ennél egy lépéssel tovább megy: megjeleníti az adott étterem címét, konyháját, átlagos értékelését, képeit és a hozzá kapcsolódó véleményeket.

Bejelentkezett állapotban további interakciós lehetőségek is megnyílnak. A felhasználó kedvencekhez adhat egy éttermet, saját értékelést írhat, illetve a profiloldalon módosíthatja preferenciáit. A rendszer ezen a ponton már valóban személyes alkalmazássá válik, nem csupán nyilvános információk gyűjteményévé.

A használhatóság fontos eleme a védett útvonalak kezelése is. A Kedvencek és a Profil oldalak csak bejelentkezett felhasználó számára érhetők el, a kliensoldali route-védelem pedig gondoskodik arról, hogy jogosulatlan látogatás esetén a rendszer a belépési oldalra irányítson vissza.

1. A látogató megnyitja a főoldalt, és eldönti, hogy böngészni vagy regisztrálni szeretne.
2. A regisztráció során megadja alapadatait és kiválaszt néhány preferenciát.
3. Belépés után az Éttermek oldalon keresést végez, illetve a térképen is átnézi a lehetőségeket.
4. Egy kiválasztott étterem részletező oldalán képeket és értékeléseket néz meg.
5. A szimpatikus helyet kedvenchez adja, majd saját véleményét is ír.
6. A Profil oldalon módosítja preferenciáit, és új ajánlásokat kér a rendszertől.

#### 4.5. Gyakori hibák és elhárításuk

Gyakori hiba, hogy a frontend elindul, de a listaoldal vagy a belépés nem működik. Ilyenkor elsőként azt kell ellenőrizni, hogy a backend valóban fut-e, és hogy a frontend által használt API-cím megegyezik-e a tényleges backend-címmel. Ha a backend más porton indul, a `VITE_API_BASE_URL` változót hozzá kell igazítani.

Szintén tipikus probléma lehet a CORS-beállításokból fakadó elérés. A repository commitjai alapján ez a projektben is külön hibajavítási pontként jelent meg, ezért ha valaki a saját környezetében módosítja a frontend futási címét, akkor szükség lehet a backend engedélyezett origin-listájának bővítésére.

Bejelentkezési gond esetén érdemes törölni a korábban eltárolt tokent és újra próbálni a folyamatot. Ha a token érvénytelen vagy a backend konfigurációja közben változott, a kliensoldali bootstrap ezt hibaként érzékelheti, és automatikusan kijelentkeztetett állapotba áll vissza.

## 5. Fejlesztői dokumentáció

### 5.1. Választott fejlesztői környezetek és eszközök

A repository alapján a projekt olyan módon épül fel, hogy többféle fejlesztői környezetben is kényelmesen kezelhető. A backend egy klasszikus .NET projektstruktúrát követ, így jól illeszkedik a Visual Studio vagy a .NET CLI-alapú munkafolyamathoz. Ezt közvetetten az is alátámasztja, hogy a commitelőzményben külön megjelenik a Visual Studio helyi állományainak kizárása. A frontend ezzel szemben Vite-alapú React projekt, amelyet tipikusan Visual Studio Code-ban vagy más modern JavaScript-szerkesztőben érdemes fejleszteni.

A fejlesztői környezet kiválasztásánál fontos szempont volt, hogy a projekt több technológiai réteget köt össze. A backendhez szükség van a C# és .NET eszköztárra, a frontendhez a Node.js ökoszisztémára, az adatbázishoz pedig elegendő a SQLite fájl jelenléte. Emiatt a projekt ideális oktatási példája annak, hogyan működik együtt egy könnyű kliens–szerver architektúra külön adatbázis-szerver nélkül.

A jelen dokumentációban szereplő fejlesztői környezet-leírás a repository szerkezetéből, a csomagleírókból és a commitüzenetekből következik. Ha a beadás során a csapat konkrétan megnevezett IDE-eket vagy kiegészítő eszközöket használt, ez a fejezet egyszerűen kiegészíthető azok felsorolásával.

| Technológia             | Réteg              | Szerep                                                                         |
|-------------------------|--------------------|--------------------------------------------------------------------------------|
| ASP.NET Core Web API    | Backend            | REST-szerű API, vezérlők, middleware, hitelesítés és hibakezelés.              |
| Entity Framework Core   | Adatelérés         | Az objektum-relációs leképezés és a migrációk kezelése.                        |
| SQLite                  | Adatbázis          | Egyszerű, fájl alapú relációs adatbázis a fejlesztéshez és a bemutatáshoz.     |
| ASP.NET Identity Core   | Felhasználókezelés | Felhasználói adatok, jelszószabályok, e-mail egyediség és hitelesítési alapok. |
| JWT Bearer              | Biztonság          | Token alapú bejelentkezés és jogosultságkezelés.                               |
| Swagger / OpenAPI       | API dokumentáció   | Fejlesztés közben az API gyors tesztelését segíti.                             |
| React 19                | Frontend           | Komponens alapú kliensoldali felület.                                          |
| Vite 7                  | Frontend build     | Gyors fejlesztői szerver és modern build folyamat.                             |
| React Router 7          | Navigáció          | Oldalak és védett útvonalak kezelése.                                          |
| TanStack Query          | Adatlekérés        | Szerverállapot-kezelés, cache és újratöltés a kliensoldalon.                   |
| Axios                   | HTTP kliens        | Központi API-kliens és interceptor alapú tokenkezelés.                         |
| Leaflet + OpenStreetMap | Térkép             | Az éttermek földrajzi megjelenítése interaktív térképen.                       |



## 5.2. Architektúra és projektstruktúra

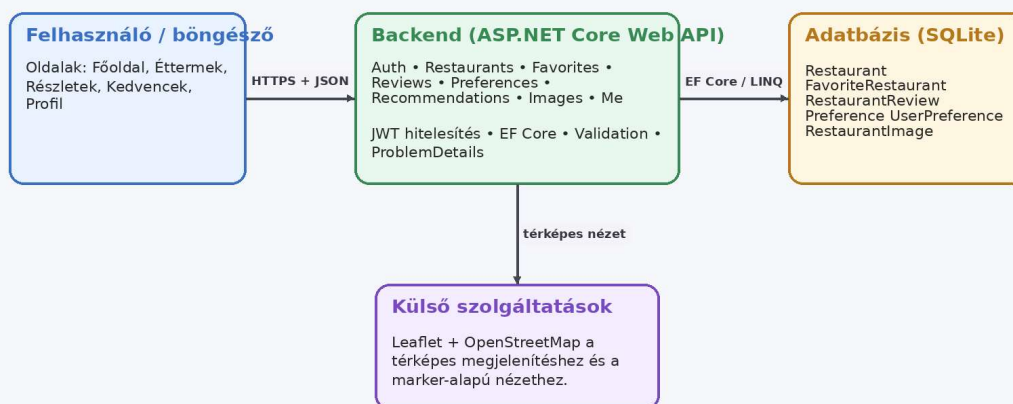
Architekturális szempontból a rendszer három fő rétegre bontható. Az első a React-alapú frontend, amely a felhasználói felületért, az útvonalkezelésért, a kliensoldali állapotokért és az API-hívásokért felel. A második az ASP.NET Core backend, amely a hitelesítést, az üzleti logikát, a validációt és az adatbázis-hozzáférést kezeli. A harmadik réteg a SQLite adatbázis, amely fájl alapú, mégis relációs szerkezetet biztosít a projekt számára.

A frontend és a backend közötti kommunikáció jól elkülönített API-modulokon keresztül történik. Ez a kliensoldalon azért előnyös, mert a React-oldalak nem szétszórt HTTP-hívásokkal dolgoznak, hanem egységes API-függvényekre támaszkodnak. A backend oldalon a vezérlők funkciók szerint vannak csoportosítva, ezért a rendszer később is viszonylag könnyen bővíthető új végpontokkal és új domain-elemekkel.

A startup konfiguráció külön erőssége, hogy a backend induláskor automatikusan előkészíti az adatbázist. Ez fejlesztési és demózási szempontból is kényelmes, mert egy frissen klónozott projekt esetén kisebb az esélye annak, hogy a hiányzó kezdeti adatok vagy migrációk miatt már az első indításnál megakadjon a bemutatás.

### Rendszerarchitektúra áttekintés

A kliensből induló kérés a React felületen át az ASP.NET Core API-hoz, majd az SQLite adatbázishoz jut.



8. ábra - Az alkalmazás architektúrájának áttekintése

## 5.3. Frontend felépítése

A frontend a Vite-alapú React projektstruktúrát követi, amely kifejezetten alkalmas kis és közepes méretű, gyorsan fejleszthető webalkalmazásokhoz. A belépési pontot a main.jsx adja, amely egyszerre húzza fel a routert, a QueryClientProvidert és az AuthProvidert. Ez azt jelenti, hogy a routing, a szerverállapot-kezelés és a felhasználói hitelesítési állapot már az alkalmazás legfelső szintjén össze van szervezve.

A src mappán belül jól elkülönülnek a felelősségi körök. A pages könyvtárban található a felhasználó által közvetlenül elért nézetek, például a főoldal, az éttermek listája, a részletező, a belépési és regisztrációs oldalak, a kedvencek, valamint a profil. A components mappa újrahaznosítható építőelemeket tartalmaz, mint a Navbar és a ProtectedRoute. Az api mappában külön modulok kezelik

az autentikációt, a kedvenceket, a képeket, a preferenciákat, az ajánlásokat, az éttermeket és az értékeléseket.

A kliensoldali hitelesítés egyik legjobb megoldása az AuthProviderben jelenik meg. Az alkalmazás indulásakor a token meglétét vizsgálja, és csak akkor próbálja lekérni a felhasználó adatait, ha valóban van tárolt token. Ez megakadályozza a felesleges hívásokat, ugyanakkor biztosítja, hogy az oldalfrissítés után is helyesen helyreálljon a bejelentkezett állapot. Az ilyen bootstrap logika tipikus problématerület modern SPA-k esetében, ezért külön érték, hogy a projektben erre önálló refaktorálási commitok is megjelentek.

A TanStack Query használata szintén tudatos technológiai döntésre utal. A komponensek nem kézzel kezelik a szerverállapot nagy részét, hanem deklaratív lekérdezéseket használnak. Ez leegyszerűsíti a töltési, hiba- és frissítési állapotok kezelését. Jól látható ez például az étterem részletező oldalon, ahol külön lekérdezések futnak az étteremadataira, a képekre és az értékelésekre, a módosító műveletek után pedig célzott cache-invalidate történik.

Külön említést érdemel a térképes nézet megvalósítása. A Restaurants oldal nem az alkalmazás indulásakor tölti be a Leaflet könyvtárat, hanem csak akkor, amikor valóban szükség van rá. Ez a dinamikus betöltés csökkenti a kezdeti terhelést, és jól mutatja, hogyan lehet külső böngészős erőforrásokat csak a szükséges oldalon használni.

| Mappa      | Feladat                                                                                            |
|------------|----------------------------------------------------------------------------------------------------|
| api        | A backenddel kommunikáló modulok, funkciók szerinti bontásban.                                     |
| components | Újrahasznosítható UI-elemek, például Navbar és ProtectedRoute.                                     |
| pages      | Oldalszintű komponensek: Home, Restaurants, Login, Register, Favorites, Profile, RestaurantDetail. |
| store      | Autentikációs kontextus és kapcsolódó segédfüggvények.                                             |
| utils      | Kisegítő modulok, például token- és storage-kezelés.                                               |

A frontend projektstruktúra egyik legfontosabb előnye, hogy jól elkülöníti az oldalszintű és a komponensszintű felelősségeket. A pages mappában a felhasználó által közvetlenül elért nézetek találhatók, míg a components mappában csak azok az elemek szerepelnek, amelyek több helyen is újrahasznosíthatók. Ez egyszerre teszi könnyebbé a karbantartást és a továbbfejlesztést.

Az api mappa funkciók szerinti felbontása különösen jó döntés. Egy központosított kliensmodul, valamint külön auth, restaurants, reviews, favorites, images, preferences és recommendations fájlok létezése azt jelzi, hogy a projekt nem közvetlen komponens-szintű fetch hívásokból nőtt ki, hanem már az első komolyabb bővítésnél felismerte az absztrakció szükségességét.

A store mappa az autentikációs állapot kezelését fogja össze. Az AuthContext, az AuthProvider és a useAuth együtt biztosítja, hogy a felhasználói állapot az egész alkalmazásban egységesen elérhető legyen. Ez egyszerűbbé teszi a jogosultságok és a bejelentkezett állapot kezelését is.

## 5.4. Backend felépítése

A backend ASP.NET Core alapú Web API, amely a domainlogikát vezérlőkbe szervezi. A Program.cs fájlban központosítva látható a teljes alkalmazásindítási folyamat: a kontrollerek, a Swagger, a

problémaleíró hibakezelés, a DbContext, az Identity Core és a JWT Bearer hitelesítés konfigurációja. Ez az egy helyre összefogott startup-struktúra átláthatóvá teszi a rendszer működését a fejlesztő számára.

A jelszókövetelmények és az egyedi e-mail cím megkövetelése azt mutatja, hogy a projekt a biztonság alapvető szempontjaira is figyelt. Bár vizsgaremek-szintről beszélünk, a kódban mégis megjelennek olyan elvek, amelyek valódi rendszerekben is fontosak: minimum hossz, szám, nagybetű és speciális karakter használata, valamint JWT alapú szerveroldali ellenőrzés.

A backend egyik technikailag erős része a CORS és az adatbázis-seedelés kezelése. A frontend fejlesztői portjai explicit módon fel vannak véve az engedélyezett eredetek közé, ami tipikus, de elengedhetetlen lépés egy külön frontend–backend fejlesztési modellnél. Ugyanilyen fontos, hogy az induláskor a rendszer migrációt futtat, majd alapadatokat seedel, így az alkalmazás kézi adatfeltöltés nélkül is demózható.

A vezérlők funkcionális bontása tiszta és könnyen követhető. Az AuthController, a MeController és a route-védelemért felelős frontend logika együtt adja az autentikációs magot. A RestaurantsController kezeli az étteremlista, a térkép és a részletező logikáját. Emellett külön controller-csoport foglalkozik a kedvencekkel, az értékelésekkel, a preferenciákkal, az ajánlásokkal és a képekkel. Ez a felosztás azért előnyös, mert a rendszer egyes felelősségei jól elkülönülnek egymástól.

A végső commitok alapján a backend stabilizálása tudatos záró feladat volt. A validáció külön mappába helyezése, a middleware megjelenése, valamint a szűrési és seedelési logika javítása azt mutatja, hogy a csapat a funkciók elkészítése után visszatért a karbantarthatóság és robusztusság kérdéséhez is. Ez egy vizsgaremek esetében kifejezetten pozitív benyomást kelt.

| Mappa / egység | Feladat                                            |
|----------------|----------------------------------------------------|
| Controllers    | A végpontok és a funkcionális API-csoportok helye. |
| Data           | DbContext és induláskori seedelés.                 |
| Entities       | Az adatbázisban tárolt domain-objektumok.          |
| Middleware     | Központi hibakezelés és keresztmetszeti logika.    |
| Validation     | Egyedi validációs attribútumok.                    |
| Migrations     | Adatbázis-migrációk.                               |

| Controller            | Felelősség                                                                     |
|-----------------------|--------------------------------------------------------------------------------|
| AuthController        | Regisztráció, bejelentkezés, kijelentkezés és token-kibocsátás.                |
| MeController          | Az aktuális felhasználó adatainak visszaadása, kliensoldali bootstrappeléshez. |
| RestaurantsController | Étteremlista, szűrés, részletező nézet, térképes listázás, konyhatípusok.      |
| FavoritesController   | Kedvencek listázása, hozzáadás és eltávolítás.                                 |
| ReviewsController     | Értékelések és vélemények lekérése, frissítése vagy felülírása.                |
| ImagesController      | Étteremképek lekérése és feltöltése.                                           |
| PreferencesController | Elérhető preferenciák és a felhasználó saját beállításai.                      |

|                           |                                                             |
|---------------------------|-------------------------------------------------------------|
| RecommendationsController | Személyre szabott étteremajánlások visszaadása.             |
| HealthController          | Egyszerű állapotellenőrzés vagy diagnosztikai célú végpont. |

A backend projektben a controller-centrikus szervezés dominál, ami ebben a méretkategóriában kifejezetten előnyös. A külön vezérlőknek köszönhetően jól elválik a hitelesítés, az étteremkezelés, a kedvencek, az értékelések, a képek és a preferenciák logikája. Ez tisztábbá teszi a forráskódot, és megkönnyíti a hibakeresést is.

Az Entities mappa egyértelműen a domainmodell tartalmazza, míg a Data mappa a DbContextet és a seedelési logikát. Ez a klasszikus felépítés megkönnyíti annak megértését, hogy mi a tartósan eltárolt adat, és mi az az infrastruktúra-réteg, amely ezeket kezeli. A Middleware és Validation mappák megjelenése azt mutatja, hogy a projekt a végére túlmutatott az „egy controllerben minden” kezdeti megoldásokon.

A külön tesztprojekt (RestaurantFinder.Api.Tests) arra is rámutat, hogy a backend rész önálló egységként kezelhető, nem csupán a frontend kiszolgálójaként. Ez fontos szemléleti előrelépés, mert a rendszer így modulárisabban vizsgálható és ellenőrizhető.

## 5.5. Adatbázis modell

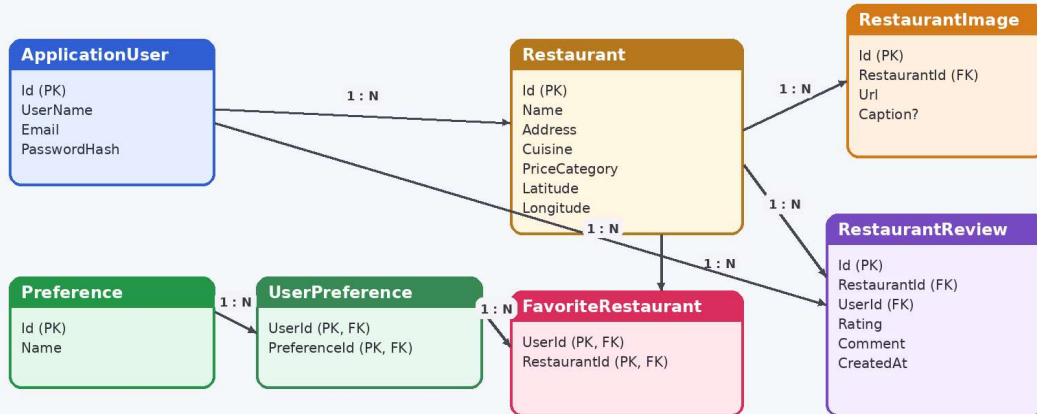
Az adatbázismodell központi elemei a felhasználók, az éttermek, a kedvencek, az értékelések, a képek és a preferenciák. A regisztrációs és profiloldal is erre a szerkezetre épül, hiszen a felhasználó preferenciái, az étteremhez tartozó értékelések és a kedvencek kezelése ugyanabba az adatkapcsolati rendszerbe illeszkedik.

A rendszer logikai felépítése jól követhető: külön entitás kezeli a felhasználót, az éttermet, az értékelést, a képet, a kedvencet és a preferenciát, valamint ezek kapcsolatait. Egy felhasználó több éttermet is elmenthet kedvencnek, több preferenciát is megadhat és több értékelést is írhat, miközben egy étteremhez több kép és több vélemény is tartozhat.

A fejlesztés során az adatmodell is fokozatosan pontosodott, ami egy ilyen projektben teljesen természetes. A végleges szerkezet már jól igazodik azokhoz a funkciókhoz, amelyek ténylegesen bekerültek az alkalmazásba.

## Aktuális logikai adatmodell

Egyszerűsített ER-szemléltetés a fő entitásokról és kapcsolataikról.



9. ábra - Az alkalmazás logikai adatmodellje

| Fő entitás         | Szerep                                                                              |
|--------------------|-------------------------------------------------------------------------------------|
| ApplicationUser    | A felhasználói fiókok alapadatai, e-mail és felhasználónév kezelésével.             |
| Restaurant         | Az éttermek törzsadatai: név, cím, koordináták, leírás, konyha, árkategória stb.    |
| RestaurantReview   | Felhasználói értékelések és szöveges vélemények.                                    |
| RestaurantImage    | Az éttermekhez kapcsolt képek és azok elérési útja / metaadatai.                    |
| FavoriteRestaurant | Kapcsolótábla a felhasználó és az általa kedvelt éttermek között.                   |
| Preference         | A választható preferenciák törzsadatai (például konyha vagy ár szerinti jelölések). |
| UserPreference     | Kapcsolótábla a felhasználó és az általa kiválasztott preferenciák között.          |

## 5.6. Kiemelt megoldások és kódrészletek

A fejlesztői dokumentációban szereplő kódrészletek célja nem a teljes forráskód lemásolása, hanem néhány olyan megoldás kiemelése, amely jól mutatja a projekt szemléletét. Ezek közül különösen fontos a központosított HTTP-kliens, a felhasználói állapot bootstrapelése, a backend induláskori konfigurációja és a térképkönyvtár dinamikus betöltése.

1. kódrészlet - Backend induláskori konfiguráció (egyszerűsített kivonat)

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
  
```

```

builder.Services.AddProblemDetails();
builder.Services.AddDbContext(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentityCore<ApplicationUser>(options =>
{
    options.User.RequireUniqueEmail = true;
    options.Password.RequiredLength = 8;
    options.Password.RequireDigit = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = true;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddSignInManager()
.AddDefaultTokenProviders();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(...);

builder.Services.AddCors(options =>
{
    options.AddPolicy("Frontend", policy =>
        policy.WithOrigins(
            "http://localhost:5173",
            "http://127.0.0.1:5173",
            "http://localhost:4173",
            "http://127.0.0.1:4173")
            .AllowAnyHeader()
            .AllowAnyMethod());
});

```

A fenti részlet azért fontos, mert szinte minden lényeges induláskori szolgáltatás ugyanitt szerveződik össze: kontrollerek, problémaalapú hibakezelés, SQLite kapcsolat, Identity, JWT és CORS. Ez a szerkezet jól bemutatathatóvá teszi az egész backend működését.

## 2. kódrészlet - Központosított Axios kliens tokenkezeléssel

```

export const api = axios.create({
    baseURL: import.meta.env.VITE_API_BASE_URL || "http://localhost:5000/api",
    withCredentials: true,
});

api.interceptors.request.use((config) => {
    const token = tokenStore.get();
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});

api.interceptors.response.use(
    (res) => res,
    (err) => {
        const msg =
            err?.response?.data?.message ||
            err?.response?.data ||
            err?.message ||
            "Hiba történt";
        return Promise.reject(new Error(msg));
    }
);

```

A kliensoldali HTTP-réteg központosítása nagyban csökkenti a kódismétlést. A token csatolása és a hibák egységes értelmezése nem az oldalszintű komponensekben, hanem egyetlen kliensmodulban történik.

### 3. kódrészlet - Felhasználói állapot bootstrapelése az AuthProviderben

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [booting, setBooting] = useState(true);

  const hydrateUser = useCallback(async () => {
    const token = tokenStore.get();
    if (!token) {
      setUser(null);
      setBooting(false);
      return;
    }
    try {
      setBooting(true);
      const me = await authApi.me();
      setUser(me);
    } catch {
      tokenStore.clear();
      setUser(null);
    } finally {
      setBooting(false);
    }
  }, []);

  useEffect(() => {
    hydrateUser();
  }, [hydrateUser]);
}
```

Ez a minta biztosítja, hogy frissítés után az alkalmazás ne veszítse el a bejelentkezett állapotot. A token megléte esetén a rendszer újra lekéri a /me jellegű felhasználói adatokat, ellenkező esetben pedig tiszta, kijelentkezett állapotból indul.

### 4. kódrészlet - Leaflet dinamikus betöltése a térképes oldalon

```
function loadLeaflet() {
  if (window.L) return Promise.resolve(window.L);

  if (!document.getElementById("leaflet-css")) {
    const css = document.createElement("link");
    css.id = "leaflet-css";
    css.rel = "stylesheet";
    css.href = "https://unpkg.com/leaflet@1.9.4/dist/leaflet.css";
    document.head.appendChild(css);
  }

  return new Promise((resolve, reject) => {
    const script = document.createElement("script");
    script.id = "leaflet-js";
    script.src = "https://unpkg.com/leaflet@1.9.4/dist/leaflet.js";
    script.async = true;
    script.onload = () => resolve(window.L);
    script.onerror = reject;
    document.body.appendChild(script);
  });
}
```

A térképkönyvtár csak azon az oldalon töltődik be, ahol ténylegesen szükség van rá. Ez fejlesztői szemmel kifejezetten jó megoldás, mert csökkenti a felesleges kezdeti terhelést.

#### 5. kódrészlet - Védett route-ok a frontendben

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/restaurants" element={<Restaurants />} />
  <Route path="/restaurants/:id" element={<RestaurantDetail />} />
  <Route path="/login" element={<Login />} />
  <Route path="/register" element={<Register />} />
  <Route
    path="/favorites"
    element={<ProtectedRoute><Favorites /></ProtectedRoute>}
  />
  <Route
    path="/profile"
    element={<ProtectedRoute><Profile /></ProtectedRoute>}
  />
</Routes>
```

A route-védelem kliensoldalon javítja a felhasználói élményt, ugyanakkor nem helyettesíti a szerveroldali ellenőrzést. Ebben a projektben a kettő együtt adja a teljesebb jogosultsági képet.

### 5.7. Biztonság, hitelesítés és validáció

A projekt biztonsági modellje a tanulási céloknak megfelelően egyszerű, ugyanakkor jó elvek mentén épül fel. A backend oldalon az Identity Core gondoskodik a felhasználói fiókok kezelésének alapjairól, a JWT Bearer hitelesítés pedig lehetővé teszi, hogy a frontend token alapú módon azonosítsa a felhasználót a további API-hívásoknál.

A kliensoldali Axios interceptor azért hasznos, mert a token csatolása nem ismétlődik minden egyes komponensben külön-külön. Ez csökkenti a hibalehetőséget, és egységesíti a kommunikációt. Hasonlóan fontos a ProtectedRoute komponens, amely felhasználói élmény szempontjából is kulturált megoldást nyújt: a védett oldalak nem hibára futnak, hanem átirányítanak a belépési felületre.

A validációs réteg külön könyvtárba szervezése arra utal, hogy a csapat a bemeneti adatok helyességét nem kizárólag kliensoldali ellenőrzésre bízta. Ez helyes irány, hiszen a végső megbízhatóságot mindig a szerveroldali validáció adja. A middleware alapú hibakezelés pedig azt segíti, hogy a felhasználó és a fejlesztő is rendezettebb hibaüzenetekkel találkozzon.

### 5.8. Preferenciák és ajánlási logika

A személyre szabott ajánlási funkció a projekt egyik legizgalmasabb eleme, mert túlmutat az egyszerű listázáson. A frontend oldalon egyértelműen látszik, hogy a felhasználó preferenciái csoportokba rendezve kezelhetők, majd a profiloldalon külön ajánlási lekérdezés kérhető. Ez arra utal, hogy a háttérben egy szabályalapú, preferencia-illesztésen alapuló ajánlási mechanizmus működik.

Bár a repositoryból nem rajzolódik ki egy bonyolult gépi tanulási rendszer, erre ebben a projektméretben nincs is szükség. Sokkal fontosabb az, hogy a felhasználó saját beállításai ténylegesen hatással legyenek a rendszer viselkedésére. A jelenlegi struktúra oktatási szempontból nagyon jó: jól



érthető, átlátható, és később könnyen továbbfejleszthető összetettebb pontozási vagy rangsorolási logikává.

## 5.9. Képkezelés és médiafunkciók

A commitelőzmény alapján a képfeltöltés külön fejlesztési szakaszként jelent meg. Ez arra utal, hogy a projekt kezdetben a szöveges és strukturált adatokra fókuszált, majd később bővült a médiaelemek kezelésével. A képek jelenléte jelentősen javítja a részletező oldal használhatóságát, ugyanakkor technikailag is összetettebbé teszi a rendszert, hiszen tárolást, listázást és kliensoldali megjelenítést egyszerre kíván.

Az alkalmazás támogatja a bináris képfájlok feltöltését, amelyet a backend oldalon az `ImagesController` valósít meg. A feltöltött állományok a szerver `wwwroot/uploads` mappájába kerülnek. A projekt futtatásához elengedhetetlen, hogy ez a könyvtár létezzen, mivel a `Program.cs`-ben konfigurált `UseStaticFiles()` middleware ezen keresztül teszi elérhetővé a fotókat a frontend számára. A képek elérési útja Guid-alapú egyedi azonosítóval kerül mentésre az adatbázisba, megelőzve ezzel a fájlnev-ütközéseket.

## 6. Az alkalmazás tesztelése és a hibák javítása

### 6.1. Tesztelési stratégia

A tesztelés ebben a projektben két jól elkülöníthető szinten jelent meg. Az első a technikai tesztelés, amit a külön backend tesztprojekt is jelez: a CustomWebApplicationFactory és a RestaurantsEndpointTests állományok arra utalnak, hogy a rendszer nem kizárólag kézi kipróbálásra épült, hanem automatizálható API-ellenőrzés is társul hozzá.

A második szint a manuális, felhasználói forgatókönyv alapú tesztelés volt. Egy ilyen alkalmazásnál különösen fontos, hogy a teljes folyamat végigpróbálható legyen a regisztrációtól és a belépéstől kezdve az étteremkeresésen, a részletező nézeteken, a kedvenceken és az értékeléseken át egészen a profiloldali ajánlásokig. A dokumentáció ezért külön teszteset-táblát is tartalmaz, amely a legfontosabb ellenőrzési pontokat foglalja össze.

A commitelőzmény több konkrét hibajavítási és finomítási pontot is megőrzött. Ezek alapján jól látható, hogy a fejlesztés során tipikus integrációs problémák is előkerültek, például CORS-beállítások, hiányzó adatmezők, auth-bootstrap kérdések, route-védelem, képkezelés és a backend végső stabilizálása. Ez azért fontos, mert a valós projektekben a munka jelentős része nem új funkciók írásából, hanem a meglévő részek összehangolásából és hibamentesítéséből áll.

A projekt tesztelési stratégiája akkor tekinthető megfelelőnek, ha három szintet lefed. Az első a build-szintű ellenőrzés: a backend fordul, a frontend csomagolható, és a fő útvonalak elindulnak. A második az API-szintű ellenőrzés: a főbb végpontok, különösen az étteremlistázás és az étterem-részletező működnek, valamint a hitelesített hívások helyesen viselkednek. A harmadik szint a felhasználói folyamatok kipróbálása: regisztráció, belépés, kedvencek, profil és értékelés.

A külön tesztprojekt jelenléte a második szintet támogatja. Bár a jelen dokumentáció nem a teljes tesztкод részletes elemzésére épül, a fájlnevek alapján jól azonosítható, hogy a fejlesztők legalább az étterem végpontok köré már elkülönített tesztelési egységet hoztak létre. Ez jó alap a későbbi bővítéshez, például autentikációs, képkezelési vagy preferencia-kezelési tesztek felé.

A manuális tesztelés ugyanakkor továbbra is elengedhetetlen maradt, mert egy ilyen vizsgaremek valódi értékét az adja, hogy a teljes felhasználói élmény következetesen működik-e. Egy webalkalmazás akkor tekinthető igazán késznek, ha a fő használati utak stabilan bejárhatóak, és a hibahelyzetek is érthetően, kulturáltan vannak kezelve.

A tesztelési fázisban kiemelt figyelmet kapott a felhasználói felület reszponzivitása. Manuális tesztekkel vizsgáltuk az étteremkereső térképes nézetét és a listát különböző kijelzőméretekben (asztali nézet, tablet és mobil). A fejlesztés során javításra került egy z-index hiba, amely korábban megakadályozta a keresőmező használatát rögzített fejlécnél. A végleges változatban a térkép és a szűrőpanel asztali nézetben rögzített (sticky), mobilnézetben pedig egymás alá rendeződve biztosítja a kényelmes használatot.

## 6.2. Manuális tesztesetek

| Azonosító | Teszt tárgya                       | Lépések röviden                                                      | Elvart eredmény                                            |
|-----------|------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------|
| TC-01     | Új felhasználó regisztrációja      | Kötelező mezők kitöltése, preferenciák kiválasztása, sikeres mentés. | A felhasználó létrejön, majd be tud jelentkezni.           |
| TC-02     | Bejelentkezés helyes adatokkal     | Érvényes e-mail és jelszó megadása.                                  | JWT token tárolódik, a védett oldalak elérhetőek lesznek.  |
| TC-03     | Bejelentkezés hibás jelszóval      | Helytelen jelszó beküldése.                                          | Érthető hibaüzenet jelenik meg, a felhasználó nem lép be.  |
| TC-04     | Étteremlista és térkép betöltése   | Éttermek oldal megnyitása.                                           | A lista és a térképes marker-készlet is megjelenik.        |
| TC-05     | Részletező oldal megnyitása        | Egy étterem kiválasztása.                                            | Megjelennek az alapadatok, képek és értékelések.           |
| TC-06     | Kedvenc hozzáadása                 | Bejelentkezett állapotban a kedvenc gomb használata.                 | Az elem megjelenik a Kedvencek oldalon is.                 |
| TC-07     | Értékelés írása                    | Pontszám és vélemény mentése.                                        | Az új értékelés rövid időn belül visszatöltődik a listába. |
| TC-08     | Profil oldali preferenciafrissítés | Preferenciák módosítása és ajánlások újratöltése.                    | A profil oldalon frissülő ajánlási lista jelenik meg.      |
| TC-09     | Kép feltöltése                     | Megfelelő formátumú kép hozzáadása egy étteremhez.                   | A kép megjelenik az étterem részletező oldalán.            |
| TC-10     | Jogosultság nélküli hozzáférés     | Védett útvonal megnyitása token nélkül.                              | A rendszer átirányít a belépési oldalra.                   |

## 6.3. A feltárt hibák és javításuk dokumentálása

| Probléma                                                        | Lehetséges ok                                             | Javítás                                                  | Forrás commit                                                                 |
|-----------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------|
| Frontend nem érte el az API-t fejlesztés közben                 | Cross-origin kérés elutasítása                            | CORS policy felvétele a frontend eredetekre              | 2026.03.10. - Add CORS policy for frontend origins                            |
| Az étteremlista nem adott vissza minden szükséges adatot        | A cím mező hiányzott a válaszmodellből                    | Address mező hozzáadása a válaszhoz                      | 2026.03.10. - Add address field to restaurant data responses                  |
| A regisztráció és a backend adatmodell eltért egymástól         | A felhasználónév mező kezelése nem volt teljesen egységes | Frontend mezőbővítés + AuthController módosítás          | 2026.03.17. - Add username field... / Update AuthController...                |
| A kliensoldali bejelentkezett állapot nehezen volt újratölthető | Bootstrap és me végpont kezelése nem volt letisztult      | Auth context refaktor, nyersebb me válasz, useAuth segéd | 2026.03.17. - Refactor auth context... / Refactor 'me'... / Create useAuth.js |

|                                                     |                                                      |                                                                   |                                                                                    |
|-----------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------|
| A térképes nézethez külön lekérdezésre volt szükség | A teljes étteremadat túl sok lehet markerrajzoláshoz | Dedikált listMapRestaurants függvény                              | 2026.03.10. - Add listMapRestaurants function                                      |
| A képkezelés hiányos volt                           | Nem volt kész feltöltési és listázási folyamat       | Új kliensoldali és szerveroldali képkezelő funkciók               | 2026.03.18. - Add function to upload images... / Implement image upload feature... |
| A backend végső stabilitása nem volt megfelelő      | Szűrés, validáció és seedelés élesítése hiányzott    | AllowedValuesAttribute rendezése, middleware, végső backend fixek | 2026.03.23. - Move AllowedValuesAttribute... / fix(backend): stabilize...          |

## 6.4. Tesztelési tanulságok

A commitok alapján látható hibajavítások azt bizonyítják, hogy a fejlesztés során folyamatos visszacsatolás történt. A projekt nem állt meg az első működő verziónál, hanem több körben javult. Különösen fontos területek voltak a frontend–backend kommunikáció, a hitelesítési állapot helyreállítása, a route-védelem, a címadatok kezelése, valamint a végső backend stabilitás. Ez a gyakorlat azt mutatja, hogy a csapat felismerte: a valódi minőséget nem csak az új funkciók száma, hanem a meglévő funkciók megbízhatósága adja.

## 7. Összegzés

---

Összességében az Étteremkereső Webalkalmazás olyan vizsgaremek lett, amely valós problémára épül, és egyszerre mutatja meg a frontend, a backend és az adatkezelés legfontosabb feladatait. A projekt erőssége, hogy nem áll meg egy egyszerű listaoldal szintjén: a hitelesítés, a kedvencek, az értékelések, a preferenciák és a térképes megjelenítés együtt már valódi webalkalmazás-jelleget adnak neki.

A fejlesztés során a rendszer lépésről lépésre állt össze. Először az alapfelületek és a backend vált működőképpé, később kerültek be a kedvencek, a profiloldal, az ajánlások, a képkezelés és a védett útvonalak, végül pedig a stabilizálás, a validáció és a seedelés pontosítása kapott nagyobb hangsúlyt.

A projekt jelenlegi állapotában már jól használható és bemutatatható, ugyanakkor több irányban is továbbfejleszthető. Új szűrési lehetőségek, részletesebb ajánlási logika, fejlettebb jogosultságkezelés vagy akár külső szolgáltatások integrálása is könnyen elképzelhető, ezért a rendszer jó alapot ad a későbbi bővítésekhez is.

## 8. Továbbfejlesztési lehetőségek

A legkézenfekvőbb továbbfejlesztési irány az ajánlási rendszer finomítása. A jelenlegi preferenciaalapú megközelítés jó kiindulás, de a jövőben érdemes lehet a korábbi értékeléseket, a kedvencek listáját, sőt akár a böngészési előzményeket is bevonni a rangsorolásba. Ezzel az alkalmazás valóban intelligensebb, személyre szabottabb szolgáltatássá válhatna.

További fontos fejlesztési lehetőség az adminisztrációs felület létrehozása. Jelenleg a rendszer elsősorban végfelhasználói nézőpontból épül fel, de egy admin modul segítségével kezelhetővé válna az éttermek adatbázisa, a feltöltött képek moderálása, a hibás vagy nem megfelelő vélemények szűrése, valamint az alapadatok karbantartása.

A térképes rész is több irányban fejleszthető. Megjelenhetne például útvonaltervezés, aktuális felhasználói pozíció figyelembevétele, távolság szerinti rendezés vagy klaszterezett markerkezelés. Ezek a kiegészítések különösen nagyobb adathalmaznál lennének hasznosak.

Frontend oldalon érdemes lenne egységes designrendszert vagy komponenskönyvtárat bevezetni, hogy a felület minden része még következetesebb legyen. Emellett hasznos lehet a formok további validációja, a hozzáférhetőség javítása, a mobilnézetek finomhangolása és részletesebb hibaüzenetek megjelenítése.

Backend oldalon további előrelépést jelentene a részletesebb naplózás, a fejlettebb tesztkészlet, az üzleti szabályok mélyebb elkülönítése szolgáltatási rétegekbe, valamint a médiafájlok külső tárolása. Ha a projekt később valósabb üzemeltetési környezetbe kerülne, akkor érdemes lenne a jelenlegi SQLite megoldást nagyobb terhelést elbíró adatbázisra cserélni.

Továbbfejlesztési szempontból fontos lenne, hogy a dokumentáció és a projektleírás a jövőben is együtt frissüljön a kóddal. Így a rendszer felépítése, az adatmodell és a megvalósított funkciók még könnyebben követhetők maradnának.

| Fejlesztési irány             | Várható haszon                                             |
|-------------------------------|------------------------------------------------------------|
| Ajánlási logika bővítése      | Jobb személyre szabás és relevánsabb találatok.            |
| Adminisztrációs felület       | Tartalomkezelés és moderáció támogatása.                   |
| Mobil és reszponzív finomítás | Szélesebb körű, kényelmesebb használat.                    |
| Részletesebb tesztkészlet     | Nagyobb stabilitás és könnyebb továbbfejlesztés.           |
| Fejlettebb térképes funkciók  | Útvonaltervezés, távolságszámítás, hely alapú rangsorolás. |
| Naplózás és monitorozás       | Könnyebb hibakeresés és üzemeltetés.                       |

## 9. Irodalomjegyzék

---

Az alábbi források a projekt technológiai hátterének, a repository feldolgozásának és a kapcsolódó fejlesztői dokumentációnak az alapját képezték.

- [1] GitHub repository – Étteremkereső Webalkalmazás - <https://github.com/foldsom/-tteremkeres-Webalkalmaz-s>
- [2] GitHub README – Étteremkereső Webalkalmazás - <https://github.com/foldsom/-tteremkeres-Webalkalmaz-s#readme>
- [3] Microsoft Learn – ASP.NET Core Web API - <https://learn.microsoft.com/aspnet/core/web-api/>
- [4] Microsoft Learn – Authentication and authorization in ASP.NET Core - <https://learn.microsoft.com/aspnet/core/security/>
- [5] Microsoft Learn – Entity Framework Core - <https://learn.microsoft.com/ef/core/>
- [6] SQLite Documentation - <https://www.sqlite.org/docs.html>
- [7] React Documentation - <https://react.dev/>
- [8] React Router Documentation - <https://reactrouter.com/>
- [9] Vite Documentation - <https://vite.dev/guide/>
- [10] TanStack Query Documentation - <https://tanstack.com/query/latest>
- [11] Axios Documentation - <https://axios-http.com/docs/intro>
- [12] Leaflet Documentation - <https://leafletjs.com/>
- [13] OpenStreetMap - <https://www.openstreetmap.org/>
- [18] MDN Web Docs - HTTP authorization header - <https://developer.mozilla.org/docs/Web/HTTP/Headers/Authorization>
- [17] Zod Documentation - <https://zod.dev/>
- [16] React Hook Form Documentation - <https://react-hook-form.com/>
- [15] Microsoft Learn - OpenAPI / Swagger támogatás ASP.NET Core környezetben - <https://learn.microsoft.com/aspnet/core/tutorials/web-api-help-pages-using-swagger>
- [14] Microsoft Learn - ASP.NET Core Identity - <https://learn.microsoft.com/aspnet/core/security/authentication/identity>

## 10. Mellékletek

A mellékletekben röviden összefoglaltam a projekt néhány fontos gyakorlati részét. Ide tartozik a mappastruktúra, a legfontosabb futtatási parancsok, a teszteléshez használható forgatókönyvek, valamint néhány kiegészítő összefoglaló is.

A mellékletek célja az is, hogy gyorsan visszakereshető legyen néhány olyan információ, amely a használat, a fejlesztés vagy a bemutatás során különösen hasznos lehet.

### 10.1. Rövid mappastruktúra

```
-tteremkeres-Webalkalmaz-s/  
├── backend/  
│   ├── RestaurantFinder.Api/  
│   │   ├── Controllers/  
│   │   ├── Data/  
│   │   ├── Entities/  
│   │   ├── Middleware/  
│   │   ├── Migrations/  
│   │   ├── Validation/  
│   │   ├── Program.cs  
│   │   └── appsettings.json  
│   └── RestaurantFinder.Api.Tests/  
│       ├── CustomWebApplicationFactory.cs  
│       └── RestaurantsEndpointTests.cs  
├── database/  
│   ├── database_schema.md  
│   └── database.txt  
└── etteremkereso-frontend/  
    ├── src/  
    │   ├── api/  
    │   ├── components/  
    │   ├── pages/  
    │   ├── store/  
    │   ├── utils/  
    │   ├── App.jsx  
    │   └── main.jsx  
    ├── package.json  
    └── vite.config.js
```

### 10.2. Gyorsindítási parancsok

```
# Backend  
cd backend/RestaurantFinder.Api  
dotnet restore  
dotnet run  
  
# Frontend  
cd etteremkereso-frontend  
npm install
```



```
npm run dev

# Opcionális környezeti változó
VITE_API_BASE_URL=http://localhost:5000/api
```

## 10.4. Kulcsfájlok gyors referencialistája

Az alábbi táblázat azt foglalja össze, hogy az egyes fontosabb funkciók mely fájlokhoz vagy mappákhoz kapcsolódnak. Ez fejlesztői szempontból gyors áttekintést ad a projekt szerkezetéről.

| Elérési út                                                        | Kategória                | Rövid szerep                                                             |
|-------------------------------------------------------------------|--------------------------|--------------------------------------------------------------------------|
| etteremkereso-frontend/src/main.jsx                               | Belépési pont            | Itt történik a Router, a Query kliens és az AuthProvider inicializálása. |
| etteremkereso-frontend/src/App.jsx                                | Útvonalkezelés           | A publikus és védett oldalak útvonalainak központi definíciója.          |
| etteremkereso-frontend/src/components/Navbar.jsx                  | Navigáció                | Felső menü, fő navigációs pontok, be- és kijelentkezési vezérlés.        |
| etteremkereso-frontend/src/components/ProtectedRoute.jsx          | Védelem                  | A hitelesített oldalak őrzése, betöltési és átirányítási logika.         |
| etteremkereso-frontend/src/pages/Home.jsx                         | Főoldal                  | Bevezető tartalom és cselekvésre ösztönző elemek.                        |
| etteremkereso-frontend/src/pages/Restaurants.jsx                  | Étteremlista             | Keresés, szűrés és térképes megjelenítés összehangolása.                 |
| etteremkereso-frontend/src/pages/RestaurantDetail.jsx             | Részletező oldal         | Étteremadatok, képek, értékelések és kedvencjelölés kezelése.            |
| etteremkereso-frontend/src/pages/Profile.jsx                      | Profil                   | Preferenciák lekérése és mentése, ajánlások megjelenítése.               |
| etteremkereso-frontend/src/api/client.js                          | HTTP kliens              | Axios alapkonfiguráció, tokenkezelés és egységes hibafeldolgozás.        |
| backend/RestaurantFinder.Api/Program.cs                           | Induláskori konfiguráció | DI, CORS, JWT, Swagger, adatbázis és seedelés beállítása.                |
| backend/RestaurantFinder.Api/Data/ApplicationDbContext.cs         | Perzisztencia            | Az EF Core entitások és kapcsolatok központi regisztrációja.             |
| backend/RestaurantFinder.Api/Data/DbSeeder.cs                     | Kezdeti adatok           | Teszteléshez és bemutatóhoz hasznos induló adatok feltöltése.            |
| backend/RestaurantFinder.Api/Controllers/RestaurantsController.cs | Fő domain API            | Éttermek listázása, részletek, térképes és szűrt lekérdezések.           |
| backend/RestaurantFinder.Api/Controllers/AuthController.cs        | Hitelesítés              | Regisztráció, belépés és kijelentkezés köré szerveződő végpontok.        |
| backend/RestaurantFinder.Api/Middleware/GlobalExceptionHandler.cs | Hibakezelés              | Egységes backend oldali kivételkezelés és válaszképzés.                  |

|                                                                |         |                                                        |
|----------------------------------------------------------------|---------|--------------------------------------------------------|
| backend/RestaurantFinder.Api.Tests/RestaurantsEndpointTests.cs | Tesztek | Példa automatizált végponttesztekre és ellenőrzésekre. |
|----------------------------------------------------------------|---------|--------------------------------------------------------|

## 10.5. Bővített kézi tesztforgatókönyvek

A következő lista olyan kézi tesztforgatókönyveket tartalmaz, amelyekkel gyorsan végigellenőrizhető a rendszer több fontos működési útvonala. Ezek segítségével jól látható, hogy a fő funkciók együtt is stabilan működnek-e.

| Teszt neve            | Lépés / bemenet                                                | Várt eredmény                                                                 |
|-----------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------|
| Első indítás          | Backend és frontend indítása alapértelmezett portokon.         | A kliens eléri az API-t, a kezdőoldal hiba nélkül betöltődik.                 |
| Regisztráció          | Új felhasználó létrehozása erős jelszóval és preferenciákkal.  | Sikeres regisztráció, majd automatikus vagy könnyű bejelentkezés.             |
| Hibás regisztráció    | Gyenge jelszó vagy duplikált e-mail megadása.                  | A rendszer érthető validációs hibát jelez.                                    |
| Bejelentkezés         | Érvényes adatokkal történő belépés.                            | A védett oldalak elérhetővé válnak, a navbar állapota frissül.                |
| Lista-szűrés          | Keresőkifejezés és több szűrő kombinálása az éttermek oldalon. | A lista szűkül, a találatok és a térképes pontok összhangban maradnak.        |
| Részletező megnyitása | Egy étterem kiválasztása a listából.                           | Megjelennek az alapadatok, képek és értékelések.                              |
| Kedvencebe mentés     | Bejelentkezett felhasználó kedvenchez ad egy éttermet.         | A kedvencek oldalon megjelenik az adott elem.                                 |
| Értékelés beküldése   | A felhasználó saját szöveges véleményt és értékelést küld.     | Az új értékelés a részletező oldalon visszaolvasható.                         |
| Profil-frissítés      | Preferenciák módosítása a profil oldalon.                      | A mentés sikeres, az ajánlások frissülnek vagy újraszámolhatók.               |
| Kijelentkezés         | A felhasználó kilép a rendszerből.                             | A védett oldalak újra csak hitelesítés után elérhetők.                        |
| Közvetlen URL-teszt   | Védett útvonal megnyitása friss böngészőlappal.                | A route-őrzés átirányít a belépési felületre vagy visszaállítja az állapotot. |
| API-hiba szimulálása  | A backend ideiglenes leállítása frontend futása mellett.       | A kliens értelmezhető hibaüzenetet ad, nem omlik össze a felület.             |

## 10.7. Részletes követelményelemzés

A következő alfejezet a korábbi leíró részeket követelményalapú, könnyen áttekinthető formában foglalja össze. Így röviden is látható, hogy a rendszer milyen funkciókat és minőségi elvárásokat teljesít.

| Azonosító | Funkcionális követelmény  | Rövid magyarázat                                                                                  |
|-----------|---------------------------|---------------------------------------------------------------------------------------------------|
| FR-01     | Regisztráció              | A rendszer új felhasználót tud létrehozni, kötelező mezők és jelszókövetelmények ellenőrzésével.  |
| FR-02     | Bejelentkezés             | Sikeres azonosítás után a felhasználó hitelesített állapotba kerül, és eléri a védett funkciókat. |
| FR-03     | Étteremlista és keresés   | A felhasználó listában és térképen is böngészheti az éttermeket, illetve szűrhet köztük.          |
| FR-04     | Étterem részletező oldal  | Egy kiválasztott étteremhez részletes adatok, képek és értékelések jelennek meg.                  |
| FR-05     | Kedvencek kezelése        | A bejelentkezett felhasználó éttermeket menthet saját listába és törölheti is azokat.             |
| FR-06     | Értékelések és vélemények | A felhasználó 1–5 skálán értékelhet, és rövid szöveges véleményt is írhat.                        |
| FR-07     | Preferenciák tárolása     | A profiloldalon beállított preferenciák menthetők és később visszatölthetők.                      |
| FR-08     | Ajánlások                 | A rendszer a preferenciák alapján személyre szabott étteremlistát tud adni.                       |

| Azonosító | Nem-funkcionális elvárás     | Megvalósítási szempont                                                                               |
|-----------|------------------------------|------------------------------------------------------------------------------------------------------|
| NFR-01    | Használhatóság               | Egyszerű navigáció, jól értelmezhető gombok, logikus oldalszerkezet és következetes visszajelzések.  |
| NFR-02    | Megbízhatóság                | Hibás bemenetekenél kezelhető válaszok, védett műveleteknél jogosultság-ellenőrzés.                  |
| NFR-03    | Karbantarthatóság            | Külön frontend és backend réteg, funkció szerinti modulbontás, tiszta API-réteg.                     |
| NFR-04    | Biztonság                    | Jelszókezelés, tokenalapú hitelesítés és védett útvonalak a biztonság alapkövetelményeihez igazodva. |
| NFR-05    | Teljesítmény és demózhatóság | Lapozás, mintaadatok és egyszerű helyi indítás támogatják a gyors kipróbálhatóságot.                 |

## 10.8. Use case-ek és felhasználói folyamatok

A felhasználói folyamatok leírása azért fontos, mert a rendszer nem önmagában funkciók halmaza, hanem egymásra épülő műveletek sorozata. Az alábbi minták a legtipikusabb használati helyzeteket foglalják össze.

### 10.8.1. Új felhasználó első belépése

A látogató a főoldalról a regisztrációs űrlapra jut, megadja felhasználónevét, e-mail címét és jelszavát, majd kiválaszthat néhány preferenciát is. Sikeres regisztráció után bejelentkezik, ezután pedig a profiloldalon már saját preferenciák és ajánlások jelennek meg számára.

| Lépés                         | Várt rendszerreakció                                                                 |
|-------------------------------|--------------------------------------------------------------------------------------|
| Főoldal megnyitása            | A rendszer áttekinthető belépési pontokat mutat a böngészéshez és a regisztrációhoz. |
| Regisztrációs űrlap kitöltése | A rendszer ellenőrzi a kötelező mezőket és a jelszó erősségét.                       |
| Sikeres regisztráció          | A felhasználó visszajelzést kap, majd beléphet a rendszerbe.                         |
| Profil megnyitása             | A preferenciákhoz kapcsolódó adatok és a személyre szabott elemek betöltődnek.       |

### 10.8.2. Étterem keresése és térképes böngészés

A felhasználó az étteremlistát megnyitva listás és térképes nézetben is láthatja az adatokat. A kereső és a szűrők segítségével gyorsan leszűkítheti a találatokat, majd egy részletező oldalra továbblépve eldöntheti, hogy elmenti-e kedvencnek az adott helyet.

### 10.8.3. Kedvencek és értékelések kezelése

Bejelentkezett állapotban a felhasználó egy éttermet kedvencekhez adhat, később eltávolíthat, illetve saját értékelést és kommentet küldhet. Ez a folyamat jól szemlélteti a védett műveletek, az állapotfrissítés és a szerveroldali perzisztencia összekapcsolását.

## 10.9. API referencia összefoglaló

Az alábbi táblák a legfontosabb végpontokat foglalják össze. Részletesebb ellenőrzéshez a futó rendszer Swagger-felülete is használható, de ez az összefoglaló a dokumentáció szempontjából a legfontosabb pontokat tartalmazza.

| Csoport | Módszer | Példa útvonal       | Rendeltetés                                        |
|---------|---------|---------------------|----------------------------------------------------|
| Auth    | POST    | /api/auth/register  | Új felhasználó regisztrálása.                      |
| Auth    | POST    | /api/auth/login     | Bejelentkezés és hitelesített állapot létrehozása. |
| Profil  | GET     | /api/preferences/me | A saját preferenciák lekérése.                     |
| Profil  | PUT     | /api/preferences/me | A saját preferenciák frissítése.                   |

| Csoport | Módszer | Példa útvonal         | Rendeltetés                                     |
|---------|---------|-----------------------|-------------------------------------------------|
| Étterem | GET     | /api/restaurants      | Lista, keresés, szűrés és lapozás.              |
| Étterem | GET     | /api/restaurants/{id} | Egy adott étterem részletes adatainak lekérése. |

|             |     |                                        |                                                         |
|-------------|-----|----------------------------------------|---------------------------------------------------------|
| Térkép      | GET | /api/restaurants/map                   | A térképes nézethez szükséges pozíciók és rövid adatok. |
| Értékelések | GET | /api/reviews/restaurant/{restaurantId} | Nyilvános értékelések és átlagok megjelenítése.         |

| Csoport     | Módszer | Példa útvonal                         | Rendeltetés                                      |
|-------------|---------|---------------------------------------|--------------------------------------------------|
| Kedvencek   | GET     | /api/favorites                        | A felhasználó saját kedvenc listájának lekérése. |
| Kedvencek   | POST    | /api/favorites/{restaurantId}         | Étterem hozzáadása a kedvencekhez.               |
| Kedvencek   | DELETE  | /api/favorites/{restaurantId}         | Étterem eltávolítása a kedvencek közül.          |
| Értékelések | POST    | /api/reviews/{restaurantId}           | Saját értékelés létrehozása vagy frissítése.     |
| Értékelések | DELETE  | /api/reviews/{restaurantId}           | Korábbi saját értékelés törlése.                 |
| Ajánlások   | GET     | /api/recommendations                  | Preferenciák alapján ajánlott éttermek lekérése. |
| Képek       | GET     | /api/images/restaurant/{restaurantId} | Az étteremhez kapcsolódó képek listázása.        |

## 10.10. Kockázatelemzés

A projekt értékeléséhez az is hozzátartozik, hogy láthatóak legyenek a rendszer kockázatai és azok a pontok, amelyek a használat vagy a továbbfejlesztés során figyelmet igényelnek. Az alábbi táblázat ezeket foglalja össze.

| Kockázat                         | Hatás                                                                                   | Kezelési javaslat                                                                        |
|----------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Környezeti eltérések futtatáskor | A frontend és a backend nem ugyanazon a porton fut, vagy a base URL eltér.              | A demó előtt ellenőrizni kell a futtatási lépéseket és a környezeti változókat.          |
| Seed adatok hiánya vagy eltérése | Az étteremlista üres, a térképes nézet nem mutat megfelelő adatot.                      | A bemutató előtt érdemes a mintaadatokat újratölteni, vagy stabil tesztadatot használni. |
| Jogosultsági hiba                | Védett oldalak vagy műveletek nem érhetők el token nélkül.                              | A védett műveleteket külön, bejelentkezett állapotban kell tesztelni.                    |
| Képek és fájlkezelés hibái       | A részletező oldal nem mutatja a médiaelemeket.                                         | A képfunkciókat érdemes külön ellenőrizni a demó előtt.                                  |
| Feladatkörök átláthatósága       | A projekt bemutatásakor nehezebb röviden összefoglalni, ki melyik résszel foglalkozott. | A feladatköröket érdemes előre egyeztetve, röviden összefoglalni.                        |

|                   |                                                            |                                                                                                                                           |
|-------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Képfájlok elérése | Az étterem adatlapján a képek nem jelennek meg (404 hiba). | Ellenőrizni kell, hogy a backend Program.cs fájljában aktív-e a UseStaticFiles() hívás, és a wwwroot/uploads mappa létezik-e és írható-e. |
|-------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|